CPSC 471 Project Proposal (G-32) — Merch Store Database System

Team Members: (Shazil Khan - 30241942, Tameem Aboueldahab - 30204890, Surkhab Mundi - 30189028)

1. Introduction

Definitions

- Merch Store: An internal catalog and ordering system where users browse items, place orders, and admins fulfill them in a timely manner.
- Catalog Item / Variant: A product (e.g., hoodie) with options like size/color (variant).
- Order: A user's request containing one or more items / products.
- Fulfillment: Admin-side process to confirm, pack, mark delivered/picked up, and update inventory.
- Inventory: Stock counts tracked per item variant.
- Database Transaction: A set of operations that must be completed together (e.g., place order + reduce inventory).

Problem summary

Many small organizations and student clubs manage merchandise orders using spreadsheets and messages, which causes inconsistent records, inventory mistakes, and poor traceability. These inconsistencies and mistakes can inflict significant financial strains on small organizations where every dollar counts, and lead to general frustration by both customers and employees/admins when items are not accurately accounted for.

Solution summary

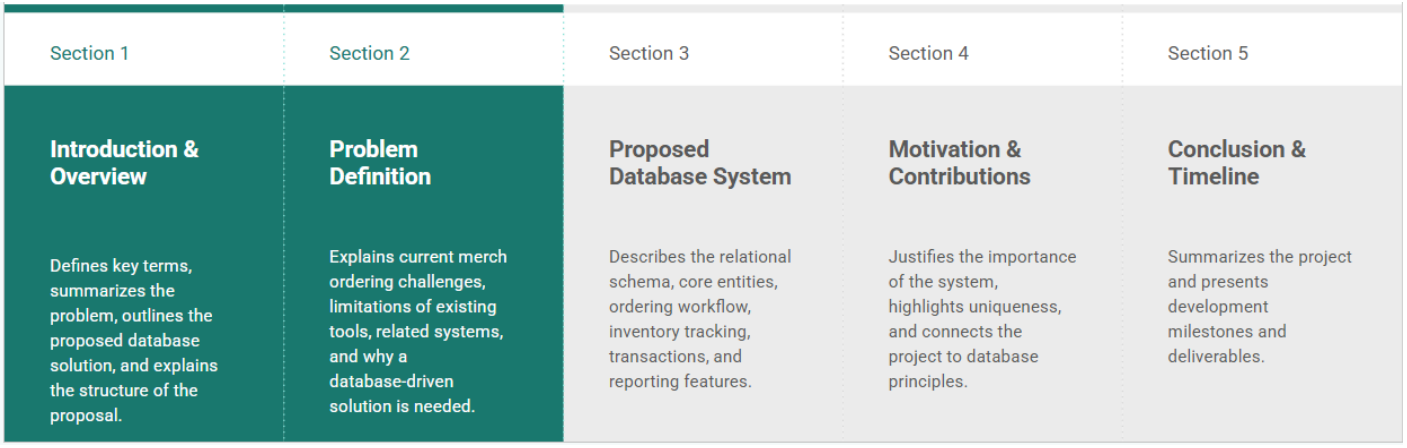
We will design and implement a relational database-backed merch store system that supports catalog management, ordering, inventory tracking, and fulfillment with strong constraints, role-based actions, and reporting queries.

Motivation

A well-designed database solution reduces errors, improves traceability, and supports reliable operations as order volume grows, which can assist small organizations and clubs that are most harmed by inconsistent and inaccurate database solutions.

Format

This proposal is organized into five main sections, beginning with a discussion of the current problem, followed by the proposed database solution, the motivation and contributions of the project, and concluding with a summary and development timeline.



---

2. Problem Definition

History / background of the problem

Merchandise ordering is commonly handled through informal tools (Google Forms, spreadsheets, email/DMs, even on paper). As order volume increases, these informal systems become increasingly difficult to maintain. Manual data entry leads to duplicated records, missing information, and inconsistent inventory counts, making it difficult to accurately track product availability or order status over time.

Why this problem is interesting

It is a real-world transactional workflow with classic database challenges:

- maintaining data integrity across related entities (ex: orders, items, variants, inventory)
- handling concurrent updates (ex: multiple users ordering at once)
- enforcing business rules (ex: stock can’t go negative)
- supporting analytics/reporting queries (ex: top items, monthly order volume, low stock)

From a database perspective, this problem highlights the importance of relational integrity, transactional consistency, and constraint enforcement. Without primary keys, foreign key relationships, and atomic updates, organizations are vulnerable to data corruption such as negative inventory, orphaned order records, and inaccurate financial totals.

When and why the problem occurs

The problem occurs when:

- inventory and orders are tracked in separate places
- multiple people update records without consistent validation
- status changes happen manually and inconsistently

Is the problem already solved? What is done now?

While large-scale e-commerce platforms provide structured solutions, they are often costly, overly complex, or unsuitable for small organizations with limited budgets and technical expertise. As a result, many groups continue relying on manual or semi-automated tools that fail to provide reliable data management.

Similar systems / solutions

- Basic e-commerce schemas like Shopify (products, orders, order items, inventory)
- Inventory + order tracking tools like Square (commercial systems exist, but they are overkill, too expensive or difficult to use, or not customizable for internal workflows)

Possible improvements over current solutions

Our project improves on spreadsheet/form workflows by:

- enforcing constraints (keys, relationships, checks)
- using transactions to preserve correctness during ordering
- enabling consistent fulfillment tracking and auditability
- producing meaningful SQL queries/views for reporting

---

3. Proposed Solution

What does this project achieve?

This project will produce a functional database-driven merch store backend that supports reliable ordering and fulfillment workflows with correct inventory updates and queryable reporting to effectively reduce inconsistencies, errors, and financial loss when used by small businesses and student clubs.

What will the project produce?

1. A normalized relational database schema (ERD + relational model)
2. SQL implementation (DDL + constraints + indexes)
3. Sample dataset (realistic test data)
4. Core SQL queries/views (reports and operational views)
5. A lightweight interface (optional depending on course expectations): CLI or simple web UI, or a set of scripted workflows demonstrating functionality

Features (relative detail)

Product & Catalog Management

- Store products with descriptions, categories, pricing
- Support variants (size/color/material, etc)
- Allow active/inactive items (discontinued products)

Entities (likely): Product, ProductVariant, Category

Inventory Tracking

- Track stock per variant
- Prevent negative stock (constraint/transaction enforcement)
- Track restock events (optional but useful)

Entities (likely): Inventory, RestockLog (optional)

Ordering Workflow

- Users can create orders containing multiple order lines
- Order totals computed from line items
- Status tracking (e.g., Pending → Approved → Fulfilled → Completed/Cancelled)
- Transaction safety: order placement + inventory decrement must succeed together

Entities (likely): Customer/User, Order, OrderLine, OrderStatusHistory (optional)

Fulfillment & Admin Operations

- Admin marks orders as fulfilled/completed
- Inventory updates are consistent with fulfillment rules
- Support exports/reporting queries for admins

Entities (likely): Admin, FulfillmentRecord (optional)
(Or role-based User table with a Role relationship.)

Reporting & Analytics Queries

Example queries/views:

- Top-selling items by quantity and revenue
- Low-stock report
- Orders by status, date range, user
- Fulfillment time analysis (if status history is tracked)

---

4. Motivation

Why do we need this solution?

Without a structured database system, organizations remain vulnerable to data inconsistencies, inventory inaccuracies, and unreliable operational reporting, which directly affect financial performance and user trust.

What makes the project unique?

Our project focuses on database quality: normalization, constraints, role separation, and transaction-safe workflows (ordering and inventory updates), plus reporting queries that match real operational needs with a special focus on small organizations and student clubs.

What the project contributes / achieves

- A complete relational design for a realistic business workflow
- Demonstration of database principles: keys, constraints, normalization, indexing, transactions, and query design
- A clear, testable system where correctness can be validated with sample scenarios
- Real-world impact on student clubs and organizations

---

5. Conclusion

This project addresses the challenges of inconsistent merchandise ordering and inventory tracking by designing a relational database system that supports structured catalog management, transactional ordering, fulfillment workflows, and reporting. By enforcing data integrity and consistency, the proposed system provides a reliable and scalable alternative to informal tracking methods.

Estimated timeline

The proposed solution demonstrates how core database management concepts can be applied to solve real-world operational problems effectively.

Jan 29: Proposal
Submit the formal proposal with a clear problem statement, planned features/workflow, and a preliminary structure capturing the main entities and relationships.

Feb 5: Proposal evaluation; scale up/down if required
Receive TA feedback and adjust scope (add/remove features or entities) to ensure the workload and complexity match course expectations.

Feb 12: Detailed ERD + assumptions (meets entity/relationship requirements)
Deliver an expanded ERD with full cardinalities/participation, keys, weak entity included, and written assumptions covering business rules (inventory, ordering, fulfillment, etc.).

Feb 26: Relational schema (tables + keys/constraints)
Convert the ERD to a logical relational model with tables, primary/foreign keys, relationship mappings, and planned constraints/indexes.

Mar 12: Working web UI draft + core DB operations
Provide a mobile-friendly web interface draft with key user/admin flows wired to core database functionality (e.g., browse, order placement, inventory updates, order status updates).

End of term: Live demo; final report + full implementation
Demonstrate the complete system live, then submit the final report (final ERD + relational model + design/implementation summary) along with source code and database scripts/data.

---

6. References

- Course textbook/notes for CPSC 471 (relational model, normalization, transactions).
- Microsoft. (n.d.). Microsoft Company Store. https://store.ecompanystore.com/microsoft/Shop/
- Microsoft. (n.d.). Xbox Gear Shop. https://xboxgearshop.com/
- Shopify. (n.d.). Managing inventory. Shopify Help Center. https://help.shopify.com/en/manual/products/inventory